

F20FO/F21FO - Digital Forensics

Lab 05: File Structure & Data Recovery

Lab 5: Objectives

This lab intends to introduce you to some core topics of data recovery (a core concept of forensics and security) and how data is stored. At the end of the lab you will be able:

1. Perform conversion between number systems
2. To describe how data is stored
3. To search and recover files from digital storage

Notes

There is nothing to submit at the end of this lab, however students are encouraged to log their results in a document. There are some questions [coloured in blue](#) that you can use as indication to record in your logbook (e.g. lab05_logbook). Often the lab instructions are intentionally open ended, and you will have to figure some things out for yourselves.

The lab will require you to install software and download additional resources from the Resources module in Canvas. Before you start reading the instructions, download the files you will need for this lab. Changes to your machine should not impact on its functionality – however it is recommended to use a Windows VM.

Lab 5.1: Number Systems

We introduced number systems with different bases in last week's lecture. Refer to the lecture materials to [complete the following conversion table](#).

Decimal	Binary (unsigned)	Hexadecimal
	11001011	
		2AF7
54		

Colours are usually defined as a mixture of Red, Green and Blue. [Complete the following table](#) for both hexadecimal and decimal RGB codes.

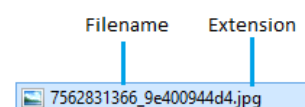
Colour	Hex code RGB	Decimal code RGB
Colour1	AB 65 2F	124 0 63
Red		
Green		
Blue		
White		

Lab 5.2: File Structure

Files are the only form of electronic data storage on computers and digital devices. It is, therefore, important to understand how they are stored and how to recover them. In digital forensics files, or parts-of files, are the digital evidence that is used to build a case and in security these are what will be either breached or used for an attack. Often, a tech savvy person can hide data by masking files and keeping them away from the naked eye.

Components of a file: A file usually comprises of 5 parts

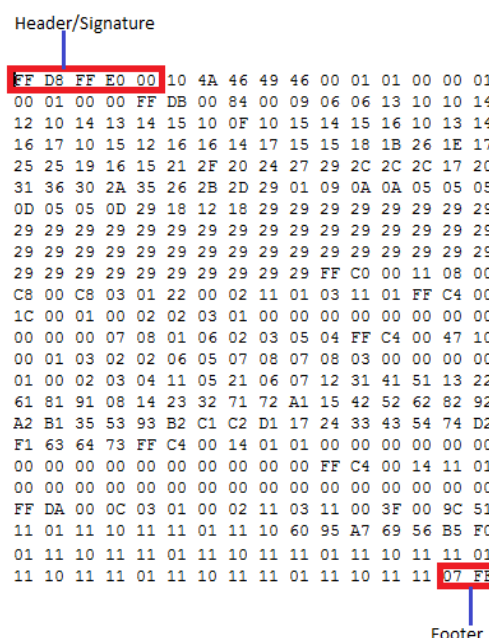
1. **Filename** - seen in your normal file explorer
2. **Extension** - seen in your normal file explorer (if not hidden)
3. **Content/Data** - depends on the file, but could be the image or content of a document, or simply the hex of the file
4. **Header/Signature** - the string at the start of the file that states what it is - only visible in hex
5. **Footer** - the end of the file that tells us when it ends (but it is not always there) - only visible in hex



This information is important because if we know how a file is stored we know what to look for to recover it.

File Signatures

Operating Systems use a process of application binding to link a file type to an application – Windows, for example, uses file extensions. The fact that Windows uses file extensions and not headers gives rise to a data-hiding technique in which the user changes the extension of the file to obscure its contents. If a file named **IllegalItem.jpg** was changed to **lansys.dll** and moved to a system folder, the casual observer would probably never find it. If a basic email scanner was in place that simply denied the sending of files with certain extensions, then a user could easily bypass such a simple system.



Linux uses a file's header (or signature) to identify which application should open the file. It is, therefore, harder to obscure a file's contents/true type as, with a broken header, the file will simply not open.

To complete the following you will need a hex editor.

We will be using **Frhed**, which you can download from the resources folder on Canvas mentioned at the start of the lab sheet. Frhed is a binary file editor for Windows that allows you to read and modify the hexadecimal of a file. The latest version can be downloaded from here: <https://sourceforge.net/projects/frhed/> (optional).

The file "File1" from your *lab05_resources* folder, is a zip file.

Unzip and open in Frhed.

The File1 is a file of unknown type (it has no extension) - but we can identify what it is from its signature (header).

Use the Gary Kessler's File Signature website:

http://www.garykessler.net/library/file_sigs.html

Try to answer the following:

What is the correct file extension for this file?

Once you have worked out the correct file extension, add the correct file extension and open the file.

What does it contain?

JPEG Signature

In the simplest cases, signatures may just be letters saying what type of files they are. In other cases, the file signature encloses other information. A JPG image file typically has the following signature

Byte	1	2	3	4	5	6	7	8	9	10	11
Hex	FF	D8	FF	E0	SKIP	SKIP	4A	46	49	46	0
Char	ÿ	ø	ÿ	à			J	F	I	F	

FF D8 represents the start of the image file (in most cases this is adequate to identify an image)

FF E0 are the application markers.

4A 46 49 46 is the JFIF (JPEG File interchange format). Again this will differ, if it uses EXIF (Exchangeable image format) - a way of storing extra information about an image (such as geo-tagging). You might see **FF E1** if the file uses EXIF.

In the files that you already downloaded from *lab05_resources* folder, there are three JPG image files (Img1.jpg, Img2.jpg, Img3.jpg).

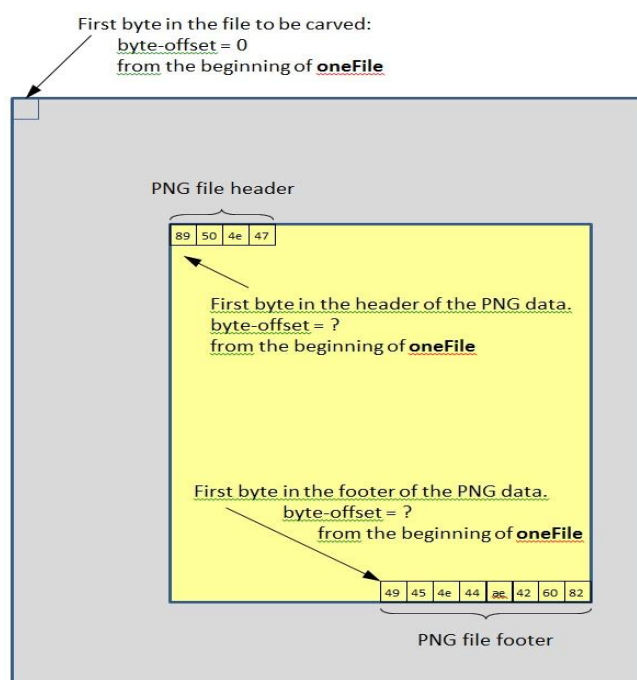
Can you identify which one was shot with a Canon camera? (Hint: use a hex editor)

Which of these images uses EXIF?

Lab 5.3: File Carving I

Understanding files allows you to recover their deleted or lost data. File carving is an incredibly useful skill to have, both in the world of computer forensics/security. Sometimes we can find sections of a disk (hard drive/usb etc) that are officially "unallocated space", meaning bytes that the OS can use to store files - but it cannot read - but actually still contain data from previous files — data that users told the OS to delete from the file system and they thought was gone! **Delete doesn't mean delete, it just means mark these bytes as reusable as needed.**

To carve a file from a block of bytes, you'll need to look for the header (the signature), and if present, the footer of the file, as depicted below:



Suppose you recover a hard drive from a suspect's computer. Your job is to find data that will help in an investigation. Now you have to perform a sequence of tasks:

To start with, let's review a basic file format. The Portable Network Graphics format give us files with a .png extension. This file type has a very distinctive header and footer.

Header in hex: 89 50 4e 47 (which reads .PNG)

Footer in hex: 49 45 4e 44 ae 42 60 82 (which reads IEND...)

- In your *lab05_resources* folder find “oneFile”.
- Using Frhed, open the “Onefile”.
- Can you see the PNG header in the file anywhere?

You can always search for PNG. Press ctrl+F and enter a search for <bh:89><bh:50><bh:4e><bh:47> to the search prompt. (the bh tells the search you are looking for hex code) You can follow the same style for the footer.

Note the location of the header and footer, so that you can easily find them again.

- Once you have the header and footer located,
 - Highlight from the header to the footer, inclusive.
 - Select Edit ⇒ Copy, and press Ok for the start and ending offsets.
 - Select File ⇒ New to create a new document.
 - Select Edit ⇒ Paste and choose the option to Insert (NOT OVERWRITE). Press ok.
 - Choose File ⇒ Save and save as a *fileone.png* file.
 - Open the file to see the image.

You have just recovered your first file - well done.

However, this was a very simple and unrealistic task, it would be unusual to have one file and such a small amount of data to scan.

Lab 5.4: File Carving II

Your next task is to carve two files from a chunk of data. In your *lab03_resources* folder find “*twoFiles*”. Open this file with Frhed.

You will use the same file carving technique with your hex editor. In this case, one file is a jpg image and the other file is an audio file of the wav format. The information below should help you on your task.

Remember, if you need to search for the hex values, use this format:

<bh:89><bh:50><bh:4e><bh:47>

File Format	Header in hex	Footer in hex
jpg	ff d8 ff e0	ff d9
wav	52 49 46 46 (RIFF)	NO FOOTER

Some file formats do not have footers. This can be problematic, but humans are often better than computers at solving this problem. In the case of the audio file, you will see a change in the information in the file - look at the pattern of data as well as large blank spaces. You can also try copying differing amounts of data into a new file, and see if it works, experimentation is the only real way to solve this.

You have just carved multiple files, something a little more realistic. But what if the data in question is megabytes large and the number of files is either very large or unknown. Because we know how to do this task on a small scale, it can be automated for larger sets of data. We'll be using a piece of software called **Scalpel**, which is free software. More information about Scalpel:

https://dfrws.org/sites/default/files/session-files/2005_USA_paper-scalpel_-_a_frugal_high_performance_file_carver.pdf

On your machine, in your workspace create a new folder and name it **Scalpel**. You can obtain a copy of Scalpel from the resources folder on Canvas mentioned at the start of the lab sheet OR alternatively (optional), download a copy of Scalpel from here [Scalpel](#) (click on the link) and extract everything under the folder **Scalpel-2.0-master** to **yourfolder\Scalpel**.

Scalpel basically automates the task of file carving for us. This file carver has a configuration file that lists many different types of files it can search for, based on headers and footers.

Go to `~\Scalpel\conf` folder and copy the file **scalpel.conf** (in the **conf** folder) into the root `~\Scalpel`. Open the `scalpel.conf` file in a text editor (e.g. notepad++); review its content, you will see that lines begin with a `#` - as in a number of programming languages a `#` is used for a comment and is therefore ignored.

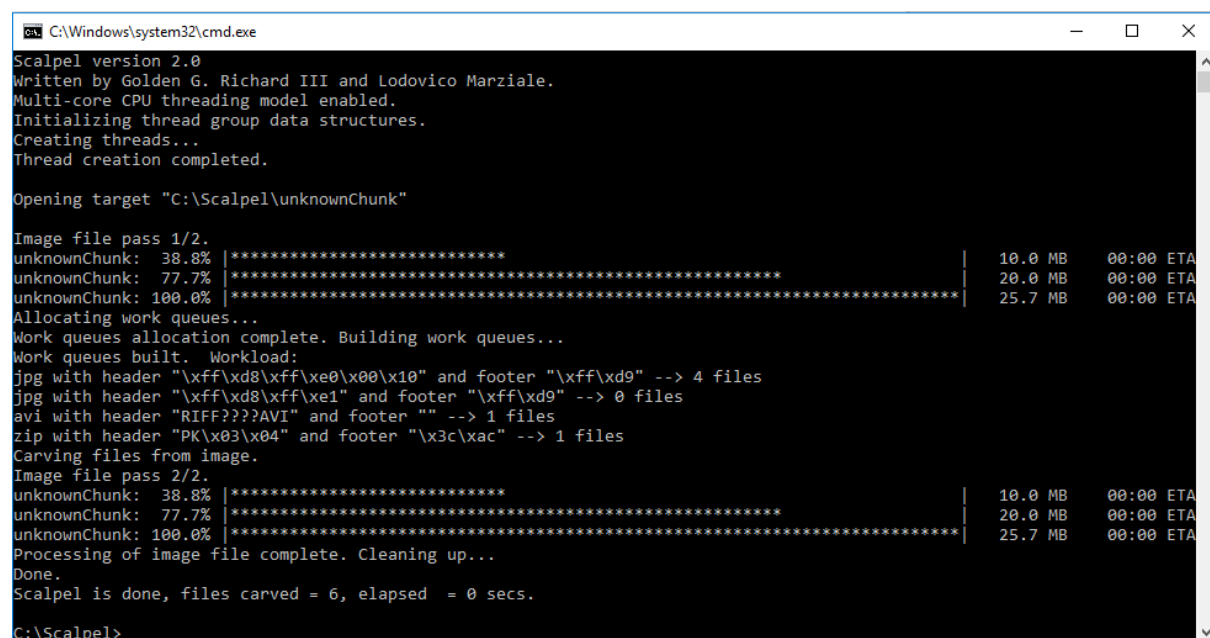
Modify the configuration file so that scalpel will search for jpg, zip, and avi files only (just remove the `#` before these file types). Review the headers these file types use.

Hint: use the jpg lines within the "GRAPHICS FILES" section; not the earlier jpg example, otherwise Scalpel gets stuck during the extraction process.

Now you are going to identify and extract an unknown quantity of data from some supplied data — follow these steps:

1. In your `lab03_resources` folder, find `unknownChunk`, copy it and save it to `~\Scalpel`.
2. Open a Command Prompt and navigate to your directory (use `cd` command).
3. Execute this command:
 - a. **Scalpel.exe unknownChunk**

You should see the following screen, as shown below:



```
C:\Windows\system32\cmd.exe
Scalpel version 2.0
Written by Golden G. Richard III and Lodovico Marziale.
Multi-core CPU threading model enabled.
Initializing thread group data structures.
Creating threads...
Thread creation completed.

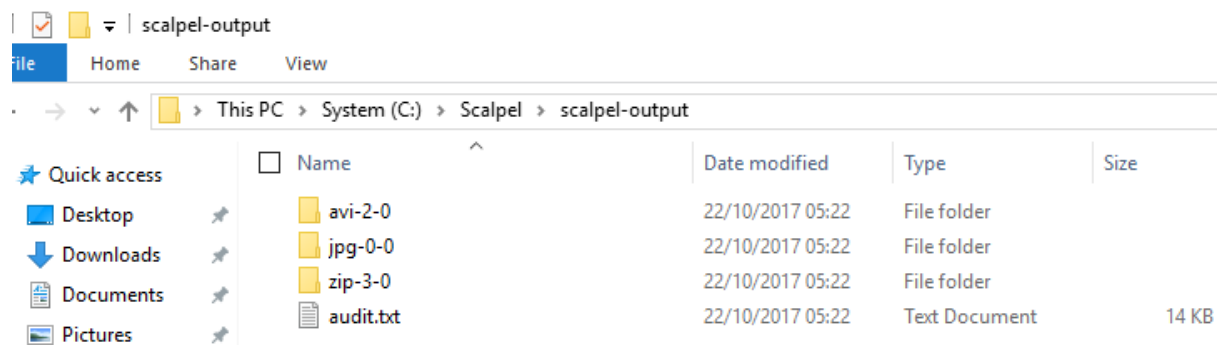
Opening target "C:\Scalpel\unknownChunk"

Image file pass 1/2.
unknownChunk: 38.8% |*****| 10.0 MB 00:00 ETA
unknownChunk: 77.7% |*****| 20.0 MB 00:00 ETA
unknownChunk: 100.0% |*****| 25.7 MB 00:00 ETA
Allocating work queues...
Work queues allocation complete. Building work queues...
Work queues built. Workload:
jpg with header "\xff\xd8\xff\xe0\x00\x10" and footer "\xff\xd9" --> 4 files
jpg with header "\xff\xd8\xff\xe1" and footer "\xff\xd9" --> 0 files
avi with header "RIFF???AVI" and footer "" --> 1 files
zip with header "PK\x03\x04" and footer "\x3c\xac" --> 1 files
Carving files from image.
Image file pass 2/2.
unknownChunk: 38.8% |*****| 10.0 MB 00:00 ETA
unknownChunk: 77.7% |*****| 20.0 MB 00:00 ETA
unknownChunk: 100.0% |*****| 25.7 MB 00:00 ETA
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 6, elapsed = 0 secs.

C:\Scalpel>
```

Data carved by Scalpel

4. A directory called **scalpel-output** will be created in your **~\scalpel** directory, as shown below.



Data carved in Scalpel output folder

5. Explore the scalpel-output directory and see what the file carver was able to retrieve
 - a. If you have any problems **look in the output folder and delete the audit file then rerun.**

NOTE: Scalpel will not run if it detects a folder called scalpel-output that already exists. If you need to run scalpel again, first delete that folder.

In your document write a short description about what you retrieved.

End of Lab Sheet